Practical 13:

Code:

```
import java.util.Scanner;

public class SubnettingDemo {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter IP address (e.g. 192.168.1.0): ");
        String ipAddress = scanner.nextLine();

        System.out.println("Choose option:");
        System.out.println("1. Calculate subnet mask based on number of subnets");
        System.out.println("2. Calculate subnet mask based on number of hosts per subnet");
        int choice = scanner.nextInt();

        if (choice == 1) {
            System.out.print("Enter number of required subnets: ");
            int subnets = scanner.nextInt();
            calculateSubnetBySubnets(ipAddress, subnets);
        } else if (choice == 2) {
            System.out.print("Enter number of required hosts per subnet: ");
            int hosts = scanner.nextInt();
            calculateSubnetByHosts(ipAddress, hosts);
        } else {
            System.out.println("Invalid choice");
        }
```

```
        scanner.close();
    }

    private static void calculateSubnetBySubnets(String ip, int subnets) {
        int bitsForSubnets = (int) Math.ceil(Math.log(subnets) / Math.log(2));
        System.out.println("Bits needed for subnetting: " + bitsForSubnets);

        int defaultMaskBits = getDefaultMaskBits(ip);
        int newMaskBits = defaultMaskBits + bitsForSubnets;
        if (newMaskBits > 32) {
            System.out.println("Subnetting not possible for given number of subnets.");
            return;
        }
        printSubnetMask(newMaskBits);

        int hostsPerSubnet = (int) Math.pow(2, 32 - newMaskBits) - 2;
        int usableSubnets = (int) Math.pow(2, bitsForSubnets);

        System.out.println("Number of usable subnets: " + usableSubnets);
        System.out.println("Number of hosts per subnet: " + hostsPerSubnet);
    }

    private static void calculateSubnetByHosts(String ip, int hosts) {
        int bitsForHosts = (int) Math.ceil(Math.log(hosts + 2) / Math.log(2));  // +2 for network and
broadcast
        System.out.println("Bits needed for hosts: " + bitsForHosts);

        int newMaskBits = 32 - bitsForHosts;
        if (newMaskBits < 0) {
```

```
        System.out.println("Invalid number of hosts.");
        return;
    }
    printSubnetMask(newMaskBits);

    int bitsForSubnets = newMaskBits - getDefaultMaskBits(ip);
    int usableSubnets = bitsForSubnets > 0 ? (int) Math.pow(2, bitsForSubnets) : 1;

    System.out.println("Number of usable subnets: " + usableSubnets);
    System.out.println("Number of hosts per subnet: " + hosts);
}

// Get default mask bits based on classful addressing
private static int getDefaultMaskBits(String ip) {
    int firstOctet = Integer.parseInt(ip.split("\\.")[0]);
    if (firstOctet >= 0 && firstOctet <= 127) {
        System.out.println("Class A network detected");
        return 8;
    } else if (firstOctet >= 128 && firstOctet <= 191) {
        System.out.println("Class B network detected");
        return 16;
    } else if (firstOctet >= 192 && firstOctet <= 223) {
        System.out.println("Class C network detected");
        return 24;
    } else {
        System.out.println("Class D or E network (multicast or experimental) - not supported.");
        return -1;
    }
}
```

```
    private static void printSubnetMask(int maskBits) {
        int mask = 0xffffffff << (32 - maskBits);

        int octet1 = (mask >> 24) & 0xff;
        int octet2 = (mask >> 16) & 0xff;
        int octet3 = (mask >> 8) & 0xff;
        int octet4 = mask & 0xff;

        System.out.println("Subnet Mask:");
        System.out.println("Decimal: " + octet1 + "." + octet2 + "." + octet3 + "." + octet4);
        System.out.println("Binary: " + toBinaryString(octet1) + "." + toBinaryString(octet2) + "." +
toBinaryString(octet3) + "." + toBinaryString(octet4));
    }

    private static String toBinaryString(int octet) {
        String s = Integer.toBinaryString(octet);
        while (s.length() < 8) {
            s = "0" + s;
        }
        return s;
    }
}
```

OUTPUT:

```
PS C:\Users\admin\Desktop\CNS>  & 'C:\Program Files\Java\jdk-25\bin\java.exe' '--enable-preview' '-
XX:+ShowCodeDetailsInExceptionMessages' '-cp'
'C:\Users\admin\AppData\Roaming\Code\User\workspaceStorage\2afde5a43ff0d48c99449cdd0a115ab
a\redhat.java\jdt_ws\CNS_859af3d6\bin' 'SubnettingDemo'
Enter IP address (e.g. 192.168.1.0):
```

192.168.1.0

Choose option:

2. Calculate subnet mask based on number of hosts per subnet

1

Bits needed for subnetting: 2

Class C network detected

Subnet Mask:

Decimal: 255.255.255.192

Binary: 11111111.11111111.11111111.11000000

Number of usable subnets: 4

Number of hosts per subnet: 62

PS C:\Users\admin\Desktop\CNS> ^C

PS C:\Users\admin\Desktop\CNS>

'C:\Users\admin\AppData\Roaming\Code\User\workspaceStorage\2afde5a43ff0d48c99449cdd0a115aba\redhat.java\jdt_ws\CNS_859af3d6\bin' 'SubnettingDemo'

Enter IP address (e.g. 192.168.1.0):

192.168.1.0

Choose option:

1. Calculate subnet mask based on number of subnets

2. Calculate subnet mask based on number of hosts per subnet

2

Enter number of required hosts per subnet: 62

Bits needed for hosts: 6

Subnet Mask:

Decimal: 255.255.255.192

Binary: 11111111.11111111.11111111.11000000

Class C network detected

Number of usable subnets: 4

Number of hosts per subnet: 62

```
PS C:\Users\admin\Desktop\CNS>
```